

GAS (Gameplay Ability System) 的 **Prediction (预测)** 机制是整个系统中**最晦涩但也最核心**的一部分。它是让多人网络游戏（尤其是动作 RPG 和 FPS）在有延迟的情况下依然感觉“顺滑”和“即时响应”的关键技术。

一句话概括：**预测就是客户端“先斩后奏”**。客户端先假装技能释放成功（播放动画、扣除蓝量），然后再告诉服务器。如果服务器同意，那就无事发生；如果服务器拒绝，客户端就“回滚”（撤销操作）。

下面我将分层为你详细教授 GAS Prediction 的原理、流程以及在代码中的应用。

一、核心概念：什么是 Prediction Key?

在 GAS 中，预测的一切都围绕着一个 ID 运转，这个 ID 叫做 **PredictionKey**。

你可以把它想象成一张**“收据”或“订单号”**。

1. **客户端发起**：当你在客户端按下按钮释放技能时，客户端的 ASC (AbilitySystemComponent) 会生成一个唯一的 PredictionKey (例如：Key=101)。
 2. **关联操作**：客户端在这个 Key 有效期内做的所有事情（扣蓝、播放特效、造成伤害），都会被打上标签：“这是订单 101 的一部分”。
 3. **发送给服务器**：客户端通过 RPC (如 ServerTryActivateAbility) 把这个 Key 发送给服务器。
 4. **服务器验证**：
 - **同意**：服务器执行技能，并告诉客户端“订单 101 批准了”。客户端收到确认后，把临时的修改变成永久的。
 - **拒绝**：服务器发现你蓝量不够或被眩晕了，告诉客户端“订单 101 作废”。客户端收到拒绝后，必须**回滚**（把扣掉的蓝加回来，停止播放动画）。
-

二、预测的生命周期 (The Flow)

让我们以你的“发射火球”为例，看看一个完整的预测流程是如何发生的：

阶段 1：客户端激活 (Local Prediction)

玩家按下鼠标左键。

1. **生成 Key**：客户端生成 PredictionKey = 250。

2. **调用 Window:** 代码中出现 FScopedPredictionWindow (即你 TargetData 代码里那个), 这标志着“记录开始”。
3. **应用副作用 (Side Effects):**
 - **Cost:** 客户端立刻扣除 50 点 Mana。此时 Mana 只是在本地变少了, 服务器还不知道。
 - **Animation:** 客户端开始播放施法蒙太奇。
 - **GameplayCue:** 客户端播放手上的火焰特效。
4. **发送 RPC:** 客户端调用 CallServerTryActivateAbility(Key=250)。

阶段 2: 服务器处理 (Server Authority)

服务器收到了 RPC。

1. **重建 Key:** 服务器拿到 Key=250, 暂时允许在这个 Key 下执行逻辑。
2. **验证条件:** 检查玩家是否有 50 Mana? 是否被眩晕? 技能是否在冷却?
3. **执行逻辑:**
 - 如果验证通过: 服务器扣除 50 Mana (这是权威数据), 生成真实的火球 Actor。
 - 如果验证失败: 服务器什么都不做。

阶段 3: 调节与回滚 (Reconciliation)

服务器将结果发回客户端。

- **情况 A: 成功 (Ack)**
 - 服务器发送 ClientActivateAbilitySucceed(Key=250)。
 - 客户端收到后, 丢掉之前的“临时修改”, 接受服务器发来的最新 Mana 数值 (通常是一样的)。玩家感觉不到任何延迟。
- **情况 B: 失败 (Reject)**
 - 服务器发送 ClientActivateAbilityFailed(Key=250)。
 - 客户端执行 **Rollback (回滚)**:
 - 把扣掉的 50 Mana 加回来。
 - 强制停止蒙太奇动画。
 - 移除火焰特效。

- 玩家会看到画面“抽搐”了一下（比如动作做了一半突然停了），这就是网络延迟带来的修正。
-

三、 我们可以预测什么？

GAS 并不是什么都能预测。为了保证同步，它只支持预测以下几类行为：

1. **Ability Activation（技能激活）**：你可以立刻开始跑技能逻辑。
2. **Attribute Modification（属性修改）**：仅限于 **Instant** 类型的 **GameplayEffect**。
 - *原理*：客户端修改的是 **CurrentValue**，而服务器同步的是 **BaseValue**。
 - *例如*：造成伤害、扣除蓝量、增加怒气。
 - *注意*：**Duration**（持续时间）和 **Infinite**（无限）类型的 **GE** **不能被预测**。你不能预测“每秒回血”的效果，只能等服务器算。
3. **GameplayCues（游戏提示）**：特效、音效。
4. **Target Data（目标数据）**：这正是你代码中 **TargetDataUnderMouse** 做的事情。你预测“我击中了这个位置”，然后发给服务器去验证。

✗ 不能预测什么？（重要）

1. **Spawn Actor（生成 Actor）**：这是最大的坑。
 - 你在客户端生成了一个 **AuraProjectile**。
 - 服务器也生成了一个 **AuraProjectile**。
 - 结果：玩家会看到两个火球（**Ghosting**）。
 - *解决方法*：通常不在客户端预测生成有逻辑的 **Projectile**。客户端只播放“挥手”动作和“发射声音”，真正的火球实体等服务器生成并复制过来。或者客户端生成一个纯视觉的“假火球”（**Fake Projectile**），等服务器真火球来了再销毁假火球。
 2. **Cooldowns（冷却时间）**：虽然 **UI** 可以显示预测的冷却，但实际的冷却逻辑必须由服务器决定。
 3. **Actor Destruction（销毁 Actor）**：你不能在客户端预测“怪死了”，必须等服务器告诉你怪死了。
-

四、代码中的 Prediction

让我们回顾你的代码，看看预测体现在哪里。

1. FScopedPredictionWindow

在你的 TargetDataUnderMouse.cpp 中：

C++

```
void UTargetDataUnderMouse::SendMouseCursorData()
{
    // 这行代码非常关键!

    FScopedPredictionWindow ScopedPrediction(AbilitySystemComponent.Get());

    // ... 打包数据 ...

    // ServerSetReplicatedTargetData 会利用上面 Window 创建的 Key
    AbilitySystemComponent->ServerSetReplicatedTargetData(...);
}
```

- **作用：**它告诉 ASC：“接下来的一段代码产生的 TargetData，请使用当前激活的 Ability 的 PredictionKey 进行标记。”
- **如果没有它：**服务器收到 TargetData 时，会发现它没有 Key，或者 Key 不匹配，从而导致服务器丢弃这份数据，技能释放失败。

2. Attribute Set 的预测处理

在属性集（AttributeSet）中，你通常会看到这样的代码（虽然你还没贴出 AttributeSet 代码，但这是标准写法）：

C++

```
void UAuraAttributeSet::OnRep_Health(const FGameplayAttributeData& OldHealth)
{
    // 这里的 true 开启了预测回滚支持

    GAMEPLAYATTRIBUTE_REPNOTIFY(UAuraAttributeSet, Health, OldHealth);
}
```

这个宏处理了复杂的逻辑：当服务器的属性值传回客户端时，它会计算“服务器的值”与“客户端预测的值”之间的差异，并平滑地修正。

五、 教学总结与最佳实践

对于你现在的项目（基于课程的 RPG）：

1. **相信服务器**：尽量把核心逻辑（伤害计算、死亡判断、生成物体）放在服务器。
2. **预测表现，不预测逻辑**：
 - 在客户端：立刻扣除 UI 上的蓝量，立刻播放挥手动画，立刻播放手掌特效。
 - 不要在客户端：直接生成具有碰撞和伤害逻辑的火球 Actor（除非你做了很复杂的假火球同步系统）。
3. **理解 TargetData 的延迟**：
 - 你的 TargetDataUnderMouse 实际上是在做一个“半预测”。客户端预测了鼠标位置，发给服务器。服务器用这个位置去生成火球。
 - 这会导致轻微的延迟：按下鼠标 -> 信号传到服务器 -> 服务器生成火球 -> 火球复制回客户端 -> 玩家看到火球飞出。
 - 对于 RPG 来说，这个延迟是可以接受的。

一句话总结 GAS Prediction：

它是一种**“乐观锁”**机制。客户端乐观地认为请求会成功，先行展示结果；如果服务器打脸（拒绝），客户端再灰溜溜地改回来。PredictionKey 就是这整个交易过程的凭证。